



DW 3

Autenticação



Autenticação

Form Login

Sistema de Vendas

Login

Usuário:

Senha:

Login



Fluxo da Autenticação

formLogin.php

Usuário digita
usuário e senha
e envia o form

AutController.php

Verifica usuário
no BD

Grava dados
em \$_SESSION

ProdutoController.php

__construct
verifica
autenticação

Se não logado
volta ao login

A sessão guarda o usuário logado e o sistema só libera o acesso às telas internas se o usuário estiver autenticado.

Sessão é uma área de memória do servidor que guarda dados do usuário entre uma página e outra (enquanto ele navega).



Tabela usuarios

```
create_usuarios.sql > ...  
1 CREATE TABLE usuarios (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     usuario VARCHAR(50) NOT NULL UNIQUE,  
4     senha VARCHAR(255) NOT NULL  
5 );  
6  
7 insert into usuarios (usuario, senha) values ("admin", "123");
```

Tabela de usuários e um usuário inserido.

Atenção: aqui a senha está em texto puro só para simplificar a aula. Em produção, nunca grave senha assim — use `password_hash()` / `password_verify()`.



formLogin.php

formLogin.php

```
1 <div class="card mt-5">
2   <div class="card-header">
3     <h5>Login</h5>
4   </div>
5   <div class="card-body">
6     <form method="post" action="AutController.php?acao=login">
7       <label class="form-label">Usuário:</label>
8       <input class="form-control" type="text"
9         name="usuario"
10        required autofocus>
11
12       <label>Senha:</label>
13       <input class="form-control" type="password"
14         name="senha"
15         required>
16       <button class="btn btn-primary mt-3"
17         type="submit">Login</button>
18     </form>
19   </div>
20 </div>
```

Form chama
o AutController
com acao=login

Inputs com
nome usuario
e senha vão
em \$_POST



AutController.php

AutController.php

```
1 <?php
2 session_start();
3 require_once 'db.php';
4
5 $acao = $_GET['acao'] ?? '';
6 switch ($acao) {
7     case 'loginForm':
8         AutController::loginForm();
9         break;
10    case 'login':
11        AutController::login();
12        break;
13    case 'logout':
14        AutController::logout();
15        break;
16 }
```

Inicia a sessão
(precisa estar
no topo do
arquivo)

Chama os
métodos
estáticos
da classe

Não precisa
instanciar:
Classe::metodo()



AutController.php — login()

```
class AutController {  
  
    public static function loginForm() {  
        include '_cabecalho.php';  
        include 'formLogin.php';  
        include '_rodape.php';  
    }  
  
    public static function login() {  
        $pdo = getConnection();  
        $stmt = $pdo->prepare("SELECT * FROM usuarios  
                                WHERE usuario = :usuario  
                                and senha = :senha");  
        $stmt->execute([':usuario' => $_POST['usuario'],  
                        ':senha' => $_POST['senha']]);  
        $usuarioDb = $stmt->fetch(PDO::FETCH_ASSOC);  
        if ($usuarioDb) {  
            $_SESSION['usuario'] = $usuarioDb['usuario'];  
            header("Location: ProdutoController.php");  
            exit;  
        } else {  
            self::loginForm();  
        }  
    }  
}
```

Método estático:
chamado sem
criar objeto

Busca no BD
o usuário e
senha digitados

Se achou:
guarda dados
em \$_SESSION
e redireciona

self:: chama
método da
própria classe



AutController.php — logout / verificar

AutController.php

```
44     public static function logout() {  
45         session_destroy();  
46         header("Location: AutController.php?acao=loginForm");  
47         exit;  
48     }  
49  
50     public static function verificarAutenticacao() {  
51         if (!self::getUsuarioLogado()) {  
52             header("Location: AutController.php?acao=loginForm");  
53             exit;  
54         }  
55     }  
56  
57     public static function getUsuarioLogado() {  
58         return $_SESSION['usuario'] ?? null;  
59     }  
60 }
```

Encerra a
sessão atual

Se NÃO há
usuário logado,
volta ao login

Retorna o
usuário da
sessão (ou
null se não
logado)



ProdutoController.php

ProdutoController.php

```
1 <?php
2 require_once 'AutController.php';
3
4 $controller = new ProdutoController();
5
6 $acao = $_GET['acao'] ?? 'index';
7 switch ($acao) {
8     case 'novo': $controller->novo();     break;
9     ...
10    default:     $controller->index();
11 }
12
13 class ProdutoController {
14
15     public function __construct() {
16         AutController::verificarAutenticacao();
17     }
18
19     public function index() { ... }
20 }
```

Carrega a
classe de
autenticação

Construtor:
roda automa-
ticamente ao
fazer
new Produto-
Controller()

Toda ação
passa primeiro
pela verificação
de login



_cabecalho.php

```
<?php if (AutController::getUsuarioLogado() != null) : ?>  
  
<div class="collapse navbar-collapse" id="navbarSupportedCo  
  
    <ul class="navbar-nav me-auto mb-2 mb-lg-0">  
  
        <li class="nav-item">  
            <a class="nav-link active" aria-current="page"  
        </li>
```

Mostra se usuário
logado

```
</div>
```

```
<?php endif; ?>
```

Fim do if



_cabecalho.php

_cabecalho.php

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head> ... </head>
4 <body>
5 <div class="container">
6   <nav class="navbar ...">
7     <div class="container-fluid">
8       <ul class="navbar-nav ...">
9         ...
10      </ul>
11
12      Usuário:
13      <?= AutController::getUsuarioLogado() ?? ' ' ?>
14      <a href="AutController.php?acao=logout">(Sair)</a>
15
16    </div>
17  </nav>
```

Mostra o
nome do
usuário logado
no topo

Link para
fazer logout



_cabecalho.php — resultado no navegador

Sistema de Vendas

Sem login: só aparece o nome do sistema (sem menu)

Sistema de Vendas

Início

Produtos

Clientes

Vendedores

Usuário: admin ([Sair](#))

Logado: menu completo (Produtos / Clientes / Vendedores) + nome do usuário + (Sair)

É exatamente o if (`AutController::getUsuarioLogado() != null`) dos slides anteriores em ação



Conceito: Sessão

Sessão é uma área de memória do servidor que guarda dados do usuário entre uma página e outra (enquanto ele navega).

`session_start()`

Liga a sessão.
Obrigatório no topo do arquivo (antes de qualquer saída).

`$_SESSION`

Array super-global onde se grava e lê os dados do usuário logado.

`session_destroy()`

Destrói a sessão (usado no logout) — o usuário precisa logar de novo.

No nosso código: `$_SESSION['usuario']` guarda quem está logado.



Métodos estáticos, self e construtor

public static function

Método da CLASSE,
não do objeto.
Chamado com
Classe::metodo() —
sem precisar de new.

self::

Dentro da classe,
chama outro método
estático da PRÓPRIA
classe. Equivale a
AutController::metodo()
mas mais curto.

__construct()

Método especial
executado AUTO-
MATICAMENTE quando
o objeto é criado:
\$c = new ProdutoController();

\$this vs self::

\$this->metodo() →
método de instância
(precisa de objeto).
self::metodo() →
método estático
(sem objeto).

Em ProdutoController, o **__construct** chama
AutController::verificarAutenticacao() — toda ação fica protegida.



Por que AutController fica visível?

ProdutoController.php

```
require_once  
'AutController.php'
```

include
_cabecalho.php

O include só
COLA
o código no
mesmo
arquivo em
execução.

AutController carregada

A classe entra no
ESCOPO GLOBAL do
PHP (fica disponível
para o resto do
request).



Por que AutController fica visível?

- `require_once` / `include` NÃO criam um arquivo separado em execução — eles juntam o código como se tudo estivesse em um único arquivo.
- Como `ProdutoController.php` carregou `AutController.php` ANTES, a classe `AutController` já está definida quando `_cabecalho.php` roda (`include`).
- Por isso `_cabecalho.php` pode usar `AutController::getUsuarioLogado()` sem precisar de `require_once` próprio.